

Lecture 19-21 - Module 5.3 Convolutional neural network

COMP 551 Applied machine learning

Yue Li
Assistant Professor
School of Computer Science
McGill University

March 18, 20, & 25, 2025

Outline

Objectives

Convolution

Padding

Stride

Dilated convolution

Multi-channels

Depthwise separable convolution

Pooling

Batch normalization

Basic ConvNet Architectures

LeNet

"We have to go deeper"

Data Augmentation

The ImageNet Dataset

AlexNet

GoogLeNet

ResNet

Image generation by backprop to the input

Application in genomics (optional)

Summary

Acknowledgement

The slides are primarily prepared based on Chapter 14 from the Murphy 2022 textbook

Learning objectives

- Basic convolution operations
- Common convolution layers
- Know how to read ConvNet architectures
- Appreciation on diverse ConvNet applications

Contents

Objectives

Convolution

Padding

Stride

Dilated convolution

Multi-channels

Depthwise separable convolution

Pooling

Batch normalization

Basic ConvNet Architectures

LeNet

"We have to go deeper"

Data Augmentation

The ImageNet Dataset

AlexNet

GoogLeNet

ResNet

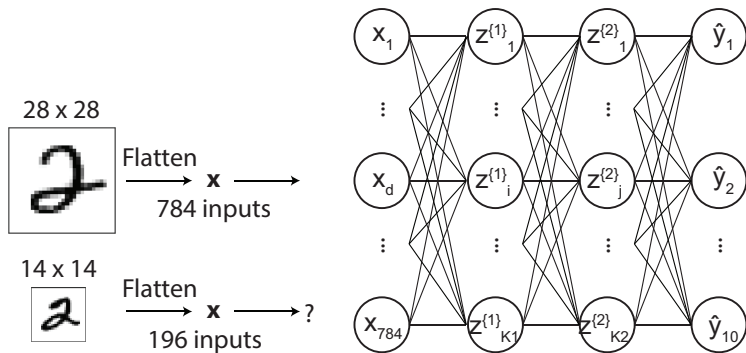
Image generation by backprop to the input

Application in genomics (optional)

Summary

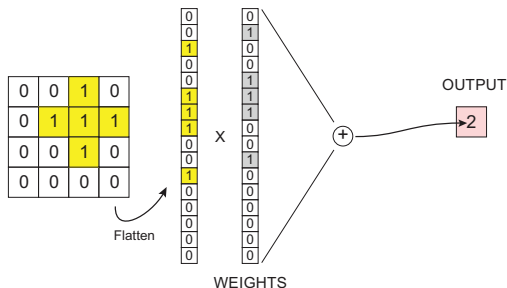
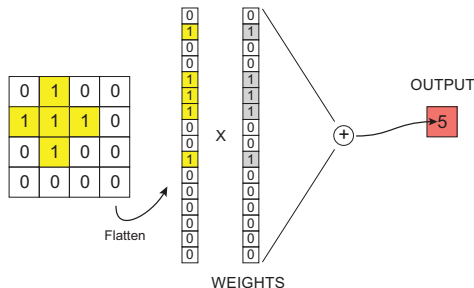
Difficulties in training MLP on unstructured data such as images

1. Input images can have variable sizes (e.g., 28×28 image, 14×14 image of "2")
2. Parameters space can be huge for high resolution images $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$, where H , W , C are height, width, and number of color channels (e.g., RGB color images)
3. A desired pattern may occur at different locations in the images. To detect the pattern regardless where it occurs, the model must be *translational invariant*.

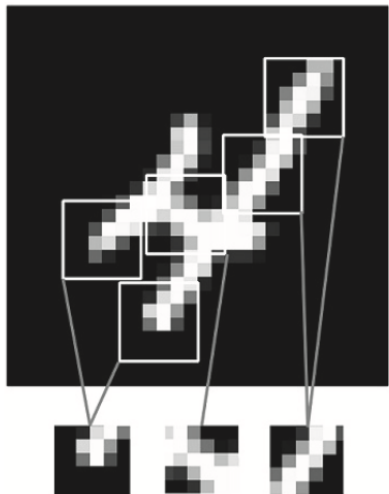


Detecting image patterns using an unstructured MLP does not work well

- Say we can design a weight vector as a **match filter** for detecting the cross-shape.
- We don't care *where* the cross-shape is but only care whether there *is* a cross-shape in the image.
- An unstructured MLP that operates on flattened images with the same weight vector would produce the highest score of 5 for the image on the left but score of 2 for the image on right although both images contain the cross-shape pattern.



Core idea of ConvNet: detecting patterns by *image templates matching*



- **Goal:** classifying if an object is present in the image *regardless its location*.
- Suppose we are building a model to predict the digit “4” in *any* image.
- What are the salient feature templates?
- How do we **learn** these templates?
- **Convolutional neural network** (ConvNet or CNN) can answer these questions.
- Note: we are *not* limited to images. We can use similar idea to classify text. [Can you think how?](#) However, there are better models for text classification (**Module 5.4**)

Illustration of 1D convolution: sliding weight vector across input vector

1d filter: $[w_0 \ w_1 \ w_2] = [7 \ 6 \ 5]$

1d input: $[x_0 \ x_1 \ x_2 \ x_3 \ x_4 \ x_5 \ x_6 \ x_7] = [0 \ 0 \ 1 \ 2 \ 3 \ 4 \ 0 \ 0]$

1d convolution at input index i : $[\mathbf{w} \circledast \mathbf{x}](i) = \sum_{u=0}^{L-1} w_u x_{i+u}$

0	0	1	2	3	4	0	0	
7	6	5	-	-	-	-	-	$z_0 = x_0 w_0 + x_1 w_1 + x_2 w_2 = 5$
-	7	6	5	-	-	-	-	$z_1 = x_1 w_0 + x_2 w_1 + x_3 w_2 = 16$
-	-	7	6	5	-	-	-	$z_2 = x_2 w_0 + x_3 w_1 + x_4 w_2 = 34$
-	-	-	7	6	5	-	-	$z_3 = x_3 w_0 + x_4 w_1 + x_5 w_2 = 52$
-	-	-	-	7	6	5	-	$z_4 = x_4 w_0 + x_5 w_1 + x_6 w_2 = 45$
-	-	-	-	-	7	6	5	$z_5 = x_5 w_0 + x_6 w_1 + x_7 w_2 = 28$

0	0	1	2	3	4	0	0	$\circledast$	7	6	5	=	5	16	34	52	45	28
---	---	---	---	---	---	---	---	---------------	---	---	---	---	---	----	----	----	----	----

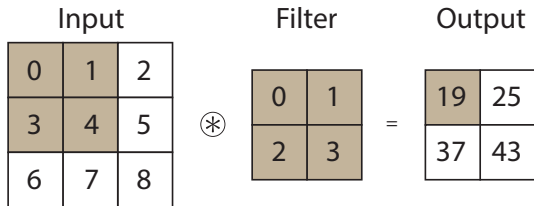
Illustration of 2D convolution

$\mathbf{W} \in \mathbb{R}^{H \times W}$ denotes a 2D filter. Convolution at the 0-based (and 1-based) pixel coordinate $[i, j]$ is:

$$[\mathbf{W} \circledast \mathbf{X}][i, j] = \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} w_{u,v} x_{i+u, j+v} \quad \left([\mathbf{W} \circledast \mathbf{X}][i, j] = \sum_{u=1}^H \sum_{v=1}^W w_{u,v} x_{i+u-1, j+v-1} \right)$$

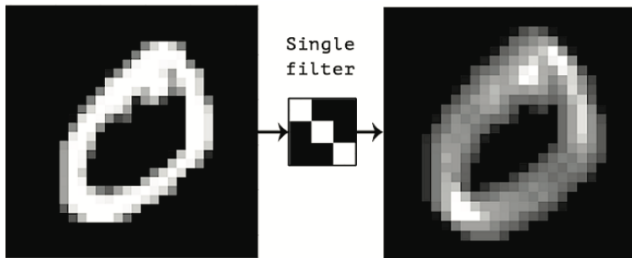
For example, convolving a 3×3 input “image” $\mathbf{X}$ with a 2×2 filter “template” $\mathbf{W}$ to compute a 2×2 output (indices are 1-based in the convolution below):

$$\begin{bmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{bmatrix} \circledast \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 x_1 + w_2 x_2 + w_3 x_4 + w_4 x_5 & w_1 x_2 + w_2 x_3 + w_3 x_5 + w_4 x_6 \\ w_1 x_4 + w_2 x_5 + w_3 x_7 + w_4 x_8 & w_1 x_5 + w_2 x_6 + w_3 x_8 + w_4 x_9 \end{bmatrix}$$



Convolution can be thought of as template matching

- The output at pixel coordinate (i, j) is large if the corresponding image patch centered at (i, j) is similar to **W**.
- Some templates correspond to orientated edges as shown in the example below: the bright spots of the feature map correspond to the pattern containing a diagonal line:



Representing 2D convolution as matrix multiplication

Because convolution is a linear operator, we can represent it by matrix multiplication by flattening the 2d matrix $\mathbf{X}$ into a 1d vector $\mathbf{x}$ and multiplying it by a matrix $\mathbf{C}$ derived from the filter $\mathbf{W}$. For example,

$$\begin{bmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{bmatrix} \circledast \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix} = \begin{bmatrix} w_1x_1 + w_2x_2 + w_3x_4 + w_4x_5 & w_1x_2 + w_2x_3 + w_3x_5 + w_4x_6 \\ w_1x_4 + w_2x_5 + w_3x_7 + w_4x_8 & w_1x_5 + w_2x_6 + w_3x_8 + w_4x_9 \end{bmatrix}$$

can be rewritten as:

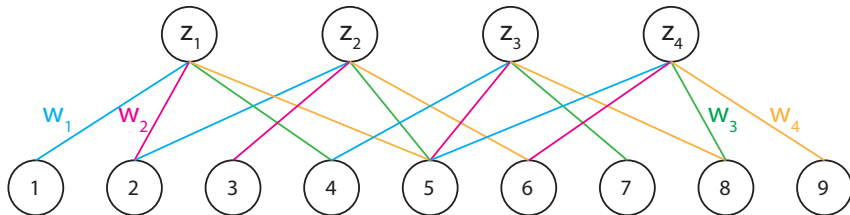
$$\begin{bmatrix} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 & x_8 & x_9 \end{bmatrix} \begin{bmatrix} w_1 & 0 & 0 & 0 \\ w_2 & w_1 & 0 & 0 \\ 0 & w_2 & 0 & 0 \\ w_3 & 0 & w_1 & 0 \\ w_4 & w_3 & w_2 & w_1 \\ 0 & w_4 & 0 & w_2 \\ 0 & 0 & w_3 & 0 \\ 0 & 0 & w_4 & w_3 \\ 0 & 0 & 0 & w_4 \end{bmatrix} = \begin{bmatrix} w_1x_1 + w_2x_2 + w_3x_4 + w_4x_5 \\ w_1x_2 + w_2x_3 + w_3x_5 + w_4x_6 \\ w_1x_4 + w_2x_5 + w_3x_7 + w_4x_8 \\ w_1x_5 + w_2x_6 + w_3x_8 + w_4x_9 \end{bmatrix}^T$$

We can recover the 2×2 output by reshaping the 4×1 vector into 2×2 .

Representing ConvNet as a partially connected MLP

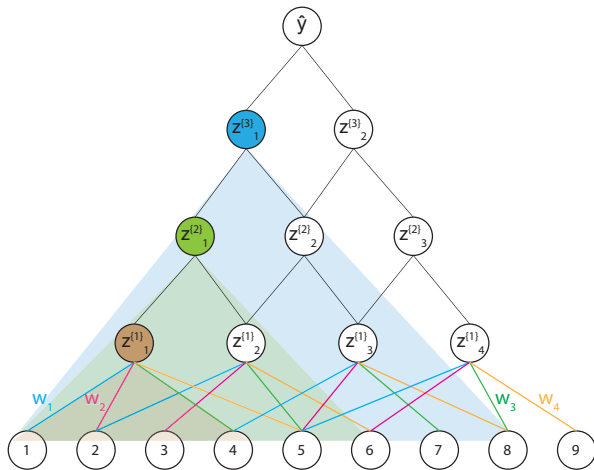
Because the weights are shared from one “image patch” to the next, we can consider the ConvNet as a *partially connected MLP* with *shared weights* as shown below.

$$\begin{bmatrix} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 & x_8 & x_9 \end{bmatrix} \begin{bmatrix} w_1 & 0 & 0 & 0 \\ w_2 & w_1 & 0 & 0 \\ 0 & w_2 & 0 & 0 \\ w_3 & 0 & w_1 & 0 \\ w_4 & w_3 & w_2 & w_1 \\ 0 & w_4 & 0 & w_2 \\ 0 & 0 & w_3 & 0 \\ 0 & 0 & w_4 & w_3 \\ 0 & 0 & 0 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 x_1 + w_2 x_2 + w_3 x_4 + w_4 x_5 \\ w_1 x_2 + w_2 x_3 + w_3 x_5 + w_4 x_6 \\ w_1 x_4 + w_2 x_5 + w_3 x_7 + w_4 x_8 \\ w_1 x_5 + w_2 x_6 + w_3 x_8 + w_4 x_9 \end{bmatrix}^T$$

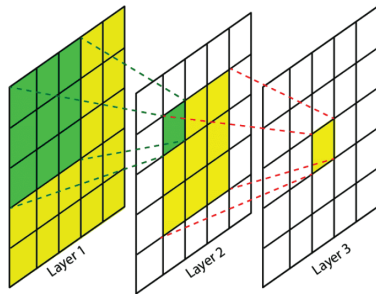


The receptive field of hidden units (“neurons”) get broader in deeper layer

MLP view



Convolution view

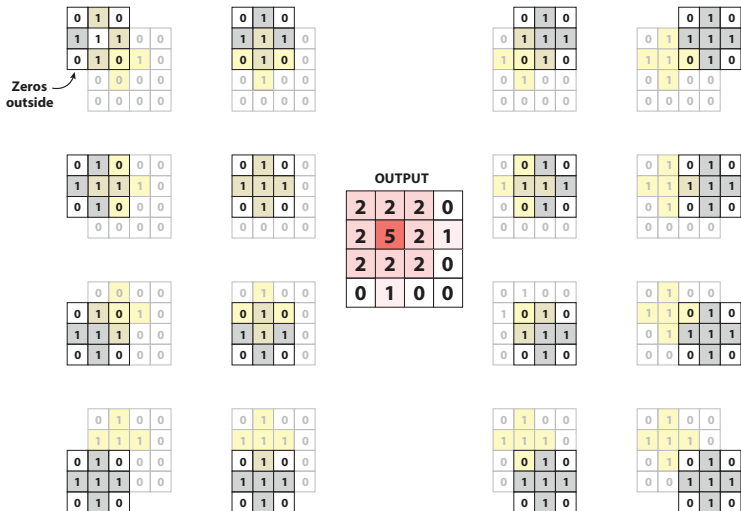


Dimension of the output as a function of the input dimension and filter size

- In the above example, convolving a 3×3 image with a 2×2 filter resulted in a 2×2 output.
- In general, convolving an $x_h \times x_w$ image with a $f_h \times f_w$ filter produces an output of size $(x_h - f_h + 1) \times (x_w - f_w + 1)$. Pop quiz: What's the output size if the filter size is the same as the input size?
- This is because we keep the filter within the image and don't slide off the edge.
- However, with this approach pixels are not used at the same frequency: the corner pixels are used only once in contrast to the center pixels.
- Also, we may want the output to have the same size as the input.
- Answer: **zero-padding**

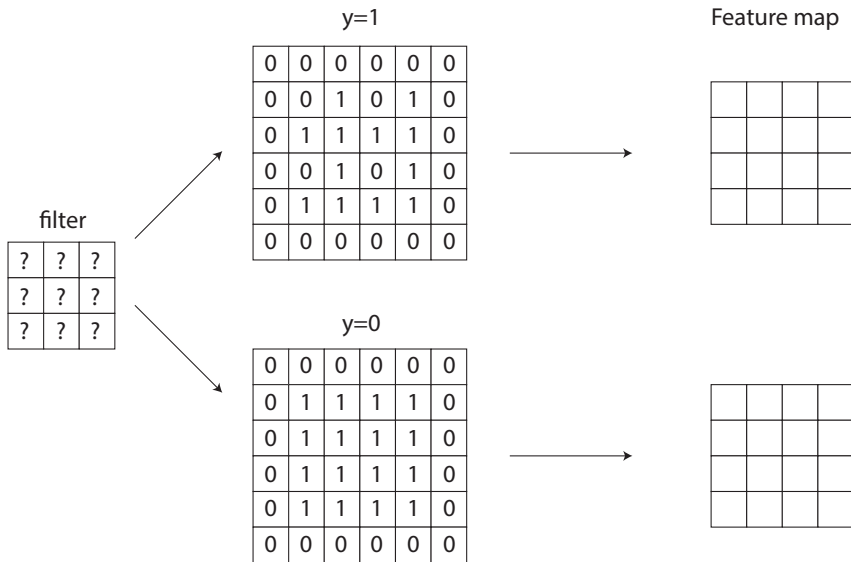
Input		Filter		Output																	
<table border="1"><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	$\otimes$	<table border="1"><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table border="1"><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

Zero-padding adds border of 0s to each side of the image



- Adding a padding of size p_h means adding p_h rows of zeros to the top and to the bottom of the image
- Adding a padding of size p_w means adding p_w columns of zeros to the left and to the right of the image
- Back to the cross-matching example on the left, we use padding $p_h = 1$ and $p_w = 1$ of each side

Pop quiz: what filter can distinguish cross from the negative image of 1's?



Output dimension as a function of padding, filter, and input size

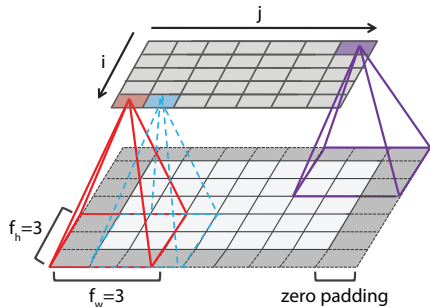
- Convolving an $x_h \times x_w$ image with $p_h \times p_w$ padding and a $f_x \times f_x$ filter produces an output of size:

$$(x_h + 2p_h - f_h + 1) \times (x_w + 2p_w - f_w + 1)$$

- In the illustration below, $p = 1, f = 3, x_h = 5, x_w = 7$, the output has size:

$$(5 + 2 - 3 + 1) \times (7 + 2 - 3 + 1) = 5 \times 7$$

Pop quiz: what padding & filter size leads to the output of the same size as the input?

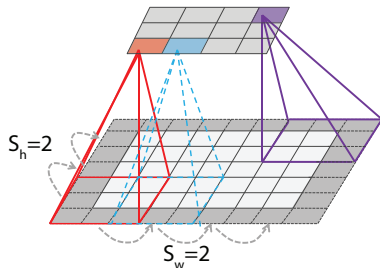
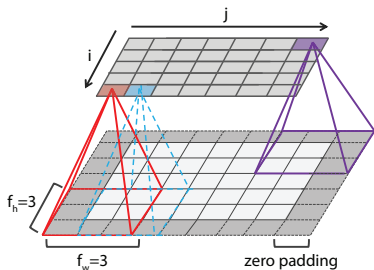


Strided convolution to reduce redundancy and speedup computation

- Neighbouring pixels are similar in values leading to similar neighbouring outputs
- We can reduce redundancy and speedup computation by *skipping* every s -th pixel:

$$z_{i,j} = b + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} x_{si+u,sj+v} w_{u,v}$$

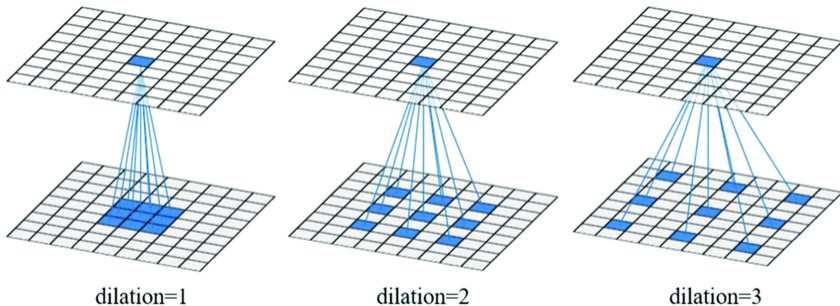
- Putting all together, the output dimension is: $\frac{x_h+2p_h-f_h+s_h}{s_h} \times \frac{x_w+2p_w-f_w+s_w}{s_w}$
- In the illustration on the right, the output dimension is calculated as follows:
 $\frac{5+2-3+2}{2} \times \frac{7+2-3+2}{2} = 3 \times 4$



Dilated convolution

- To increase receptive field of each neuron, we can go deeper by stacking convolution layers with stride.
- Alternatively, dilated convolution takes every r -th input element when performing convolution, where r is known as the rate or dilation factor:

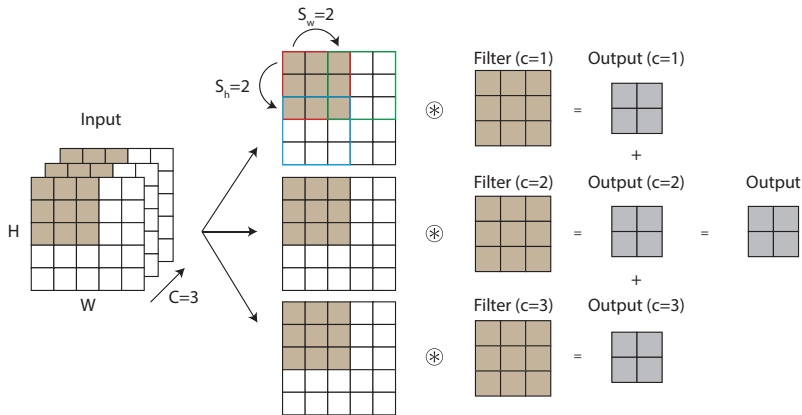
$$z_{i,j} = b + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} x_{i+ru,j+rv} w_{u,v}$$



Multi-channel filter to scan multi-channel input images

Typical input images have multiple channels (RGB). We can have a 3D weight matrix or **tensor** $\mathbf{W} \in \mathbb{R}^{H \times W \times C}$ to convolve each channel and then **sum over channels**:

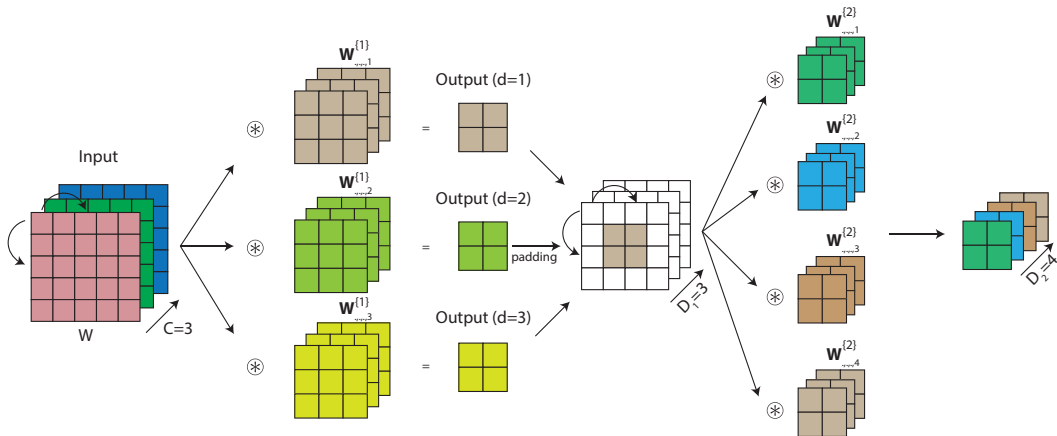
$$z_{i,j} = b + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} \sum_{c=0}^{C-1} x_{si+u, sj+v, c} w_{u,v, c}$$



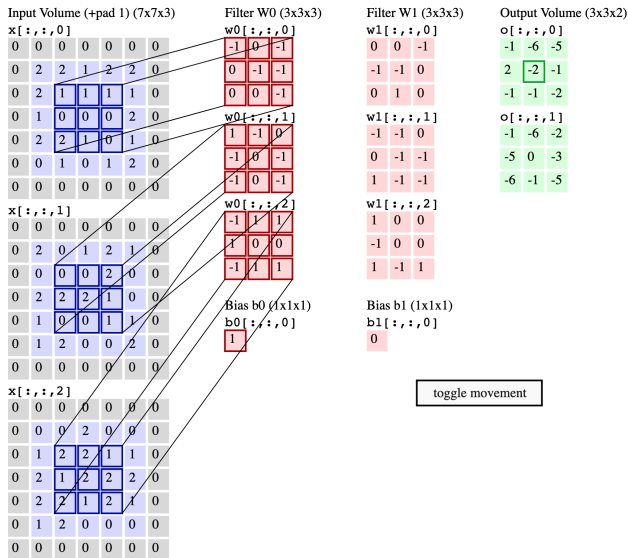
Multi-channel outputs enables mapping from $D_{\ell-1}$ to D_ℓ channels

Each channel is produced by convolving the input with $\mathbf{W}_{\dots,d}^{\{\ell\}} \in \mathbb{R}^{H \times W \times C}$:

$$z_{i,j,d}^{\{1\}} = \sum_{u=0}^{H_1-1} \sum_{v=0}^{W_1-1} \sum_{c=0}^{C-1} x_{si+u,sj+v,c} w_{u,v,c,d}^{\{1\}}, \quad z_{i,j,d'}^{\{2\}} = \sum_{u=0}^{H_2-1} \sum_{v=0}^{W_2-1} \sum_{d=0}^{D_1-1} z_{si+u,sj+v,d}^{\{1\}} w_{u,v,d,d'}^{\{2\}}$$



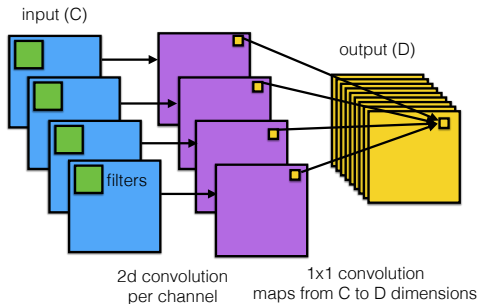
Stanford Convolution demo



Depthwise separable convolution (aka lightweight ConvNet)

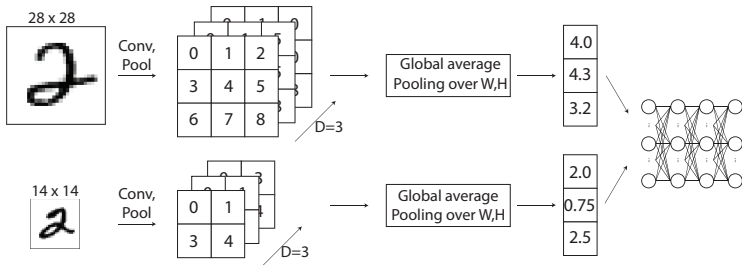
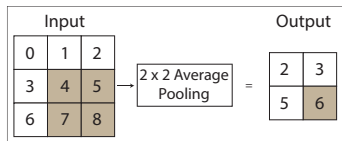
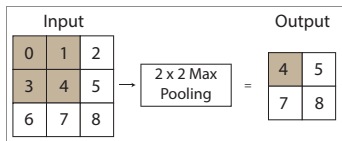
- Standard convolution uses a filter of size $H \times W \times C \times D$, which can be huge and requires a lot of data to learn
- We can *decouple* the dimensions by sharing a $H \times W$ 2D filter $\mathbf{W}$ across all C input channels and D output channels and have another $C \times D$ weight filter $\mathbf{W}'$:

$$z_{i,j,d} = b_d + \sum_{c=0}^{C-1} w'_{c,d} \left(\sum_{u=0}^{H-1} \sum_{v=0}^{W-1} x_{i+u,j+v,c} w_{u,v} \right)$$



Pooling reduces dimension of the feature map

- Max (average) pooling computes the maximum (average) over a defined patch
- Global pooling converts a $H \times W \times D$ feature map into a $1 \times 1 \times D$ dimensional feature map, which can be passed into a *fully connected layer* for prediction.
- Global pooling also allows us to apply the ConvNet to an image of *different sizes*

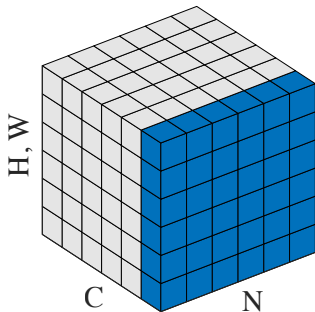


Batch normalization (BN) layers

- The activations of samples in a minibatch are standardized to have zero mean and unit variance:

$$\mu = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{z} \in \mathcal{B}} \mathbf{z}, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{z} \in \mathcal{B}} (\mathbf{z} - \mu_{\mathcal{B}})^2, \quad \hat{\mathbf{z}}_n = \frac{\mathbf{z}_n - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad \tilde{\mathbf{z}}_n = \gamma \cdot \hat{\mathbf{z}}_n + \beta$$

- Here the word “normalization” means standardization. Indeed, applying BN to the input layer is equivalent to the standardization procedure in **Module 4.1**.
- The parameters γ and β are learnable parameters.
- At test time, we compute $\mu^{\{\ell\}}$ and $\sigma^{\{\ell\}}$ for layer ℓ across *all* training examples and then use those as the learned parameters to standardize test data.
- BN reduces sensitivity to learning rate and reduces gradient explosion.



Outline

Objectives

Convolution

- Padding

- Stride

- Dilated convolution

- Multi-channels

- Depthwise separable convolution

- Pooling

- Batch normalization

Basic ConvNet Architectures

LeNet

"We have to go deeper"

Data Augmentation

The ImageNet Dataset

AlexNet

GoogLeNet

ResNet

Image generation by backprop to the input

Application in genomics (optional)

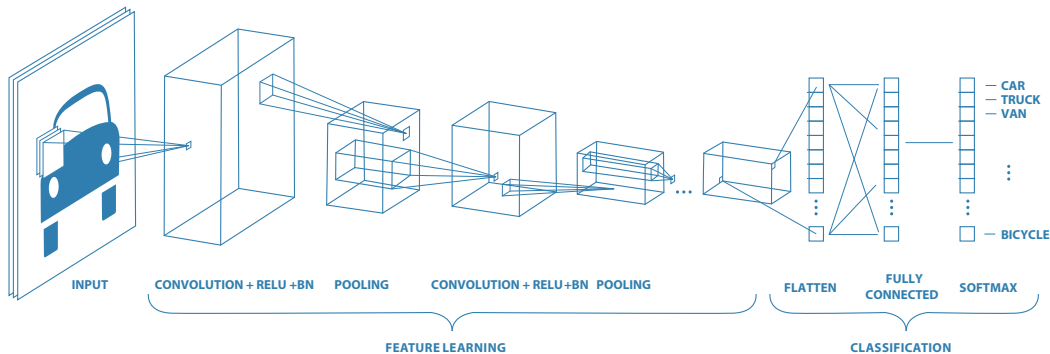
Summary

Putting it all together:

$$\{\text{convolution} + \text{pooling} + \text{BN}\}_L + \text{flatten} + \{\text{FC}\}_{L'} + \text{softmax}$$

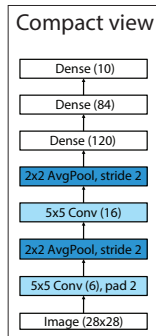
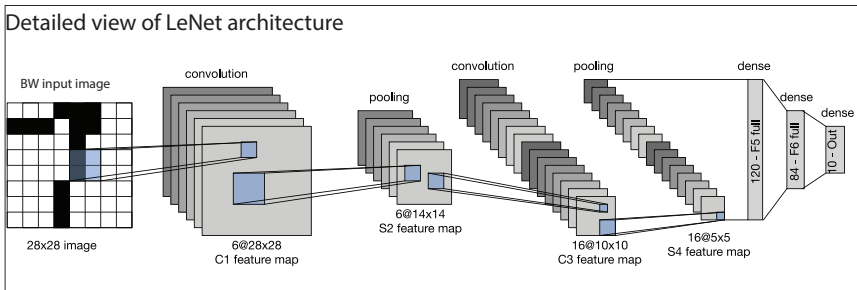
As shown below, a common ConvNet architecture has:

1. alternating layers between $\{\text{convolution} + \text{ReLU} + \text{BN}\}$ and $\{\text{max pooling}\}$,
2. flatten or max pool layer to collapse the weight and height to get a vector,
3. several fully connected layers,
4. and final softmax outputs for classification.

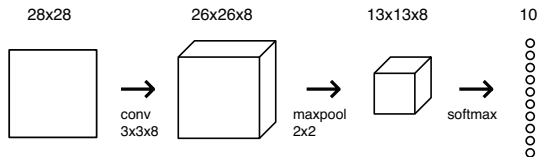


LeNet [LeCun 1998] "Gradient-Based Learning Applied to Document Recognition". In: IEEE 86.11 2278-2324

- As one of the earliest ConvNet, created in 1998, named after its creator Yann LeCun, LeNet was designed to classify hand-written digits in the MNIST dataset
- After just 1 epoch, the test accuracy is already 98.8%. By contrast, a 2-layer MLP (128 hidden each) had an accuracy of 95.9% after 1 epoch.
- More rounds of training can further increase accuracy to nearly perfect.
- Extended LeNet was used to predict postal codes in the US.



A basic ConvNet implemented using Tensorflow Keras library



```
1 from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten
2
3 num_filters = 8
4 filter_size = 3
5 pool_size = 2
6
7 model = Sequential([
8     Conv2D(num_filters, filter_size, input_shape=(28, 28, 1)),
9     MaxPooling2D(pool_size=pool_size),
10    Flatten(),
11    Dense(10, activation='softmax'),
12 ])
```

Compile and hit that “fit” button (colab)

```
1  # Compile the model (i.e., generate a DAG ready for autodiff and backprop)
2  model.compile(
3      'adam',
4      loss='categorical_crossentropy',
5      metrics=['accuracy'],
6  )
7
8  # Train the model.
9  model.fit(
10     train_images,
11     to_categorical(train_labels),
12     epochs=3,
13     validation_data=(test_images, to_categorical(test_labels)))
```

Output:

```
1  Epoch 1
2  loss: 0.2433 - acc: 0.9276 - val_loss: 0.1176 - val_acc: 0.9634
3  Epoch 2
4  loss: 0.1184 - acc: 0.9648 - val_loss: 0.0936 - val_acc: 0.9721
5  Epoch 3
6  loss: 0.0930 - acc: 0.9721 - val_loss: 0.0778 - val_acc: 0.9744
```

Outline

Objectives

Convolution

- Padding

- Stride

- Dilated convolution

- Multi-channels

- Depthwise separable convolution

- Pooling

- Batch normalization

Basic ConvNet Architectures

LeNet

"We have to go deeper"

Data Augmentation

The ImageNet Dataset

AlexNet

GoogLeNet

ResNet

Image generation by backprop to the input

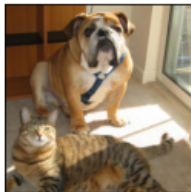
Application in genomics (optional)

Summary

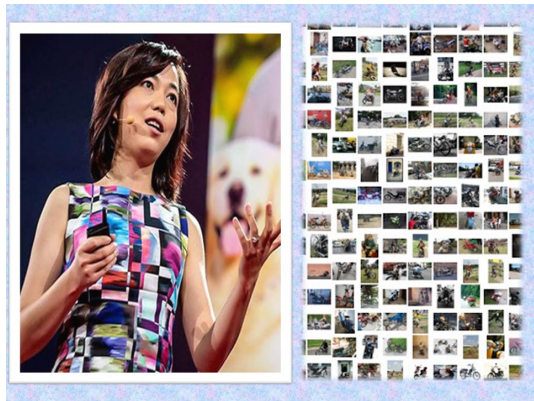
Data Augmentation

The biggest concern in training large and deep neural networks is *overfitting* due to insufficient labelled training data. We can artificially augment the dataset using label-preserving transformations. There are multiple ways of data augmentation:

1. Generating image translations and horizontal reflections.
2. Extracting random patches (224×224 used by AlexNet) (and their horizontal reflections) from the original images (256×256 in the image datasets AlexNet was applied) and training the network on these extracted patches
3. Altering the intensities of the RGB channels in training images
4. This increases the size of our training set by a factor of 2048, though the resulting training examples are, of course, highly inter-dependent.



The ImageNet Dataset (<https://www.image-net.org/>)



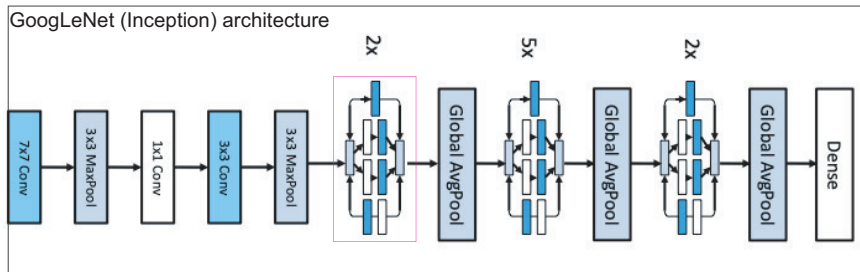
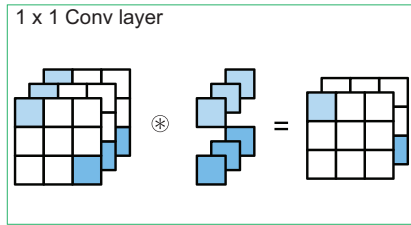
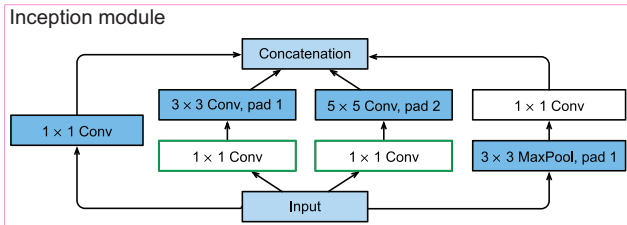
[image source](#)

Finding enough labeled data has always been a challenge for Deep Learning community. In 2009 Feifei Li, a professor at Princeton (at Stanford), launched ImageNet which is a database of **14 million labeled images containing >20K categories** – *hand-annotated* via a massive crowdsourcing website called [Amazon Mechanical Turk](#). It served as a benchmark for the machine learning researchers who would participate in ImageNet Large Scale Visual Recognition Challenge (ILSVRC) every year.

More details on ImageNet on [Wikipedia](#)

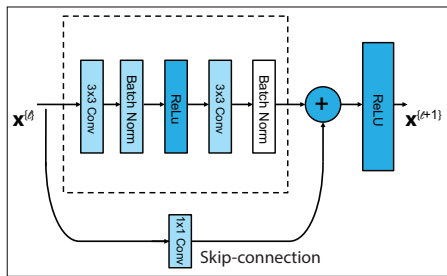
ImageNet took Feifei Li over two years to create with her postdoc. It was long and hard efforts under skepticism by others at the time. To learn more, check out her recent book: “The Worlds I See”.

Inception layer lets GoogLeNet (2014 winner) decide what filter size to use



Pop quiz: What kind of filter can simply pass the image as it is to the next layer?

ResNet from MSR (2015 Winner in ImageNet challenge)

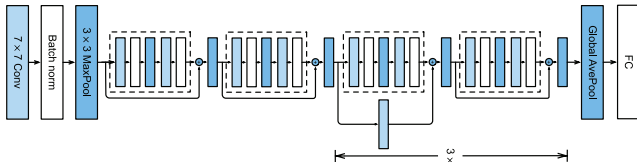


- Residual block (as shown on the left):

$$\mathbf{x}^{\{l+1\}} = \varphi(\mathbf{w} \circledast \mathbf{x}^{\{l\}} + \mathcal{F}_l(\mathbf{x}^{\{l\}}))$$

- Residual block allows training *very deep* model with >1000 layers without overfitting!
- However, the ReLU activation function φ may attenuate the signal from the current layer even when $\mathcal{F}_l(\mathbf{x}^{\{l\}}) = 0$
- A modified residual block called Preactivation Resnet (PreResnet):

$$\mathbf{x}^{\{l+1\}} = \mathbf{w} \circledast \mathbf{x}^{\{l\}} + \varphi(\mathcal{F}_l(\mathbf{x}^{\{l\}}))$$



Outline

Objectives

Convolution

- Padding

- Stride

- Dilated convolution

- Multi-channels

- Depthwise separable convolution

- Pooling

- Batch normalization

Basic ConvNet Architectures

LeNet

"We have to go deeper"

Data Augmentation

The ImageNet Dataset

AlexNet

GoogLeNet

ResNet

Image generation by backprop to the input

Application in genomics (optional)

Summary

Converting a trained classifier into an image generative model

- Goal: generate images based on a label, e.g., given the label c as “ostrich”, we would like to generate many ostrich images that are *not* in the training data.
- Idea: by Bayes’ rule, we can sample from the posterior distribution of the input:

$$p(\mathbf{x}|y = c) \propto p(\mathbf{x})p(y = c|\mathbf{x})$$

- At iteration t , we take derivatives w.r.t. the input pixels:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \epsilon_1 \frac{\partial \log p(\mathbf{x}_t)}{\partial \mathbf{x}_t} + \epsilon_2 \frac{\partial \log p(y = c|\mathbf{x}_t)}{\partial \mathbf{x}_t} + \mathcal{N}(0, \epsilon_3^2 \mathbf{I})$$

- The likelihood $p(y = c|\mathbf{x})$ is set to be *pre-trained* ConvNet (e.g., AlexNet)
- The simplest image prior is $p(\mathbf{x}) = \mathcal{N}(\mathbf{x}|\mathbf{0}, \mathbf{I})$
- A more realistic prior is to regularize the differences between adjacent pixels to reduce total variation (TV): $p(\mathbf{x}) \propto -\prod_{i,j,c} (x_{i,j,c} - x_{i+1,j,c})^2 + (x_{i,j,c} - x_{i,j+1,c})^2$

Images generated by the same pre-trained ConvNet model with priors

Gaussian prior:



goose



ostrich

TV prior:



Anemone Fish



Banana



Parachute

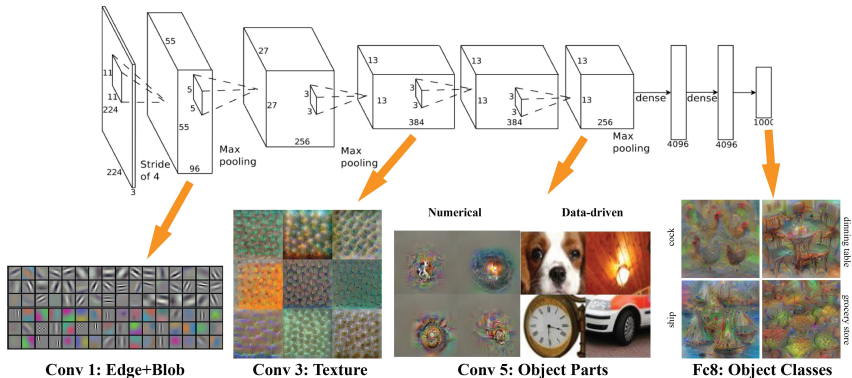


Screw

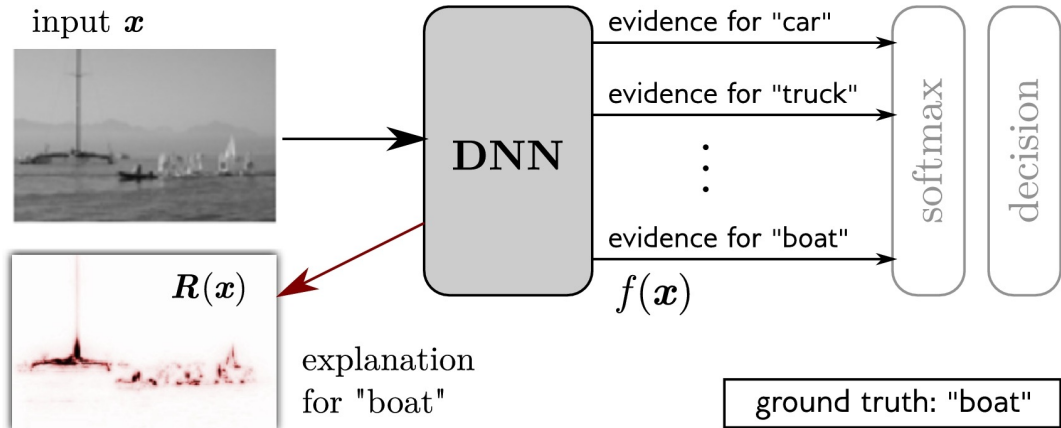
Look into the “blackbox”: visualize the features learned by the ConvNet

- Goal: we want to know what “neurons” in the trained ConvNet are learning
- Idea: similar to the image generation, we can maximize the activation of a particular neuron at layer ℓ w.r.t. **the input pixels**:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \epsilon_1 \frac{\partial \log p(\mathbf{x}_t)}{\partial \mathbf{x}_t} + \epsilon_2 \frac{\partial \log p(\mathbf{z}_i^{\{\ell\}} | \mathbf{x}_t)}{\partial \mathbf{x}_t} + \mathcal{N}(0, \epsilon_3^2 \mathbf{I})$$



Explainable AI (xAI) on individual predictions



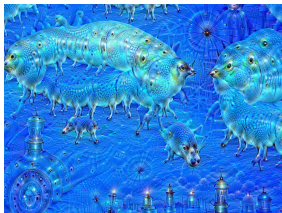
Montavon, G., Samek, W., & Müller, K. R. (2018). Methods for interpreting and understanding deep neural networks. Digital signal processing, 73, 1-15.

Going “artistic” with DeepDream

- Goal: we want to let the ConvNet generate different versions of the input images
- Idea: we can amplify the activation by maximizing the *energy* of the hidden activities at each layer

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \epsilon_1 \frac{\partial \log p(\mathbf{x}_t)}{\partial \mathbf{x}_t} + \epsilon_2 \sum_{\ell=1}^L \sum_{i=1}^{K_\ell} \frac{\partial \log p(z_i^{\{\ell\}} | \mathbf{x}_t)}{\partial \mathbf{x}_t} + \mathcal{N}(0, \epsilon_3^2 \mathbf{I})$$

- Illustrated below shows how the GoogLeNet Inception turns a Jelly fish image (left panel) into a creepy-looking images of hallucinations of dog parts after 10 (middle panel) and 50 iterations (right panel), since it was trained on many kinds of dogs.



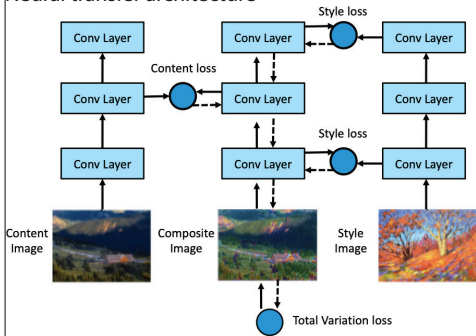
Neural style transfer

- Goal: Re-render image $\mathbf{x}$ from the *content image* $\mathbf{x}_c$ based on the *style image* $\mathbf{x}_s$
- Idea: Minimizing a joint loss w.r.t. the new content image $\mathbf{x}$:

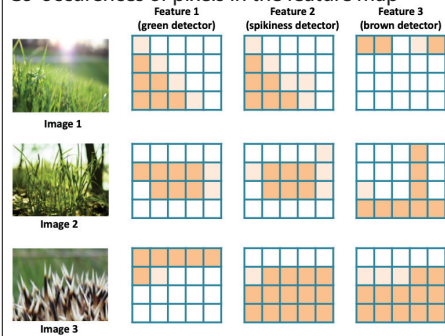
$$\mathcal{L}(\mathbf{x}|\mathbf{x}_s, \mathbf{x}_c) = \lambda_{TV}\mathcal{L}_{TV}(\mathbf{x}) + \lambda_c\mathcal{L}_{content}(\mathbf{x}, \mathbf{x}_c) + \lambda_s\mathcal{L}_{style}(\mathbf{x}, \mathbf{x}_s)$$

where $\mathcal{L}_{TV} = -\log p(\mathbf{x}_c)$ is the prior loss as above; $\mathcal{L}_{content}(\mathbf{x}, \mathbf{x}_c) = \|\phi^{\{\ell\}}(\mathbf{x}) - \phi^{\{\ell\}}(\mathbf{x}_c)\|_2^2$ is the difference in the feature maps between $\mathbf{x}$ and $\mathbf{x}_c$; $\mathcal{L}_{style}^{\{\ell\}}(\mathbf{x}, \mathbf{x}_s) = \|G^{\{\ell\}}(\mathbf{x}) - G^{\{\ell\}}(\mathbf{x}_s)\|_2^2$ is the difference in pixel *co-occurrence* in the feature maps between $\mathbf{x}$ and $\mathbf{x}_s$

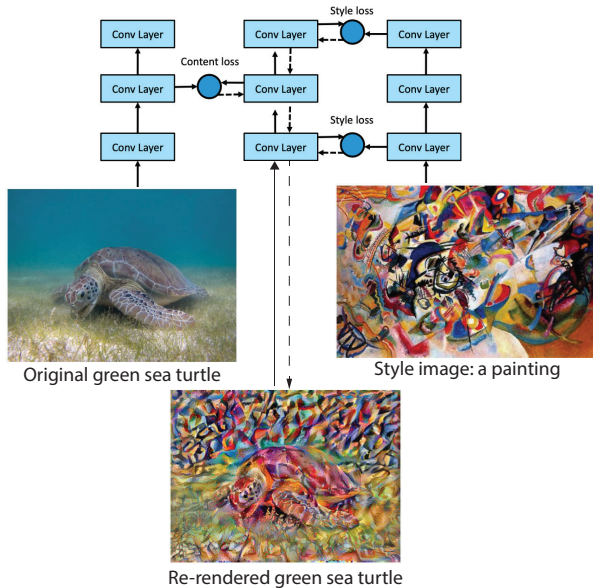
Neural transfer architecture



Co-occurrences of pixels in the feature map



Re-rendered images with different styles



Outline

Objectives

Convolution

Padding

Stride

Dilated convolution

Multi-channels

Depthwise separable convolution

Pooling

Batch normalization

Basic ConvNet Architectures

LeNet

"We have to go deeper"

Data Augmentation

The ImageNet Dataset

AlexNet

GoogLeNet

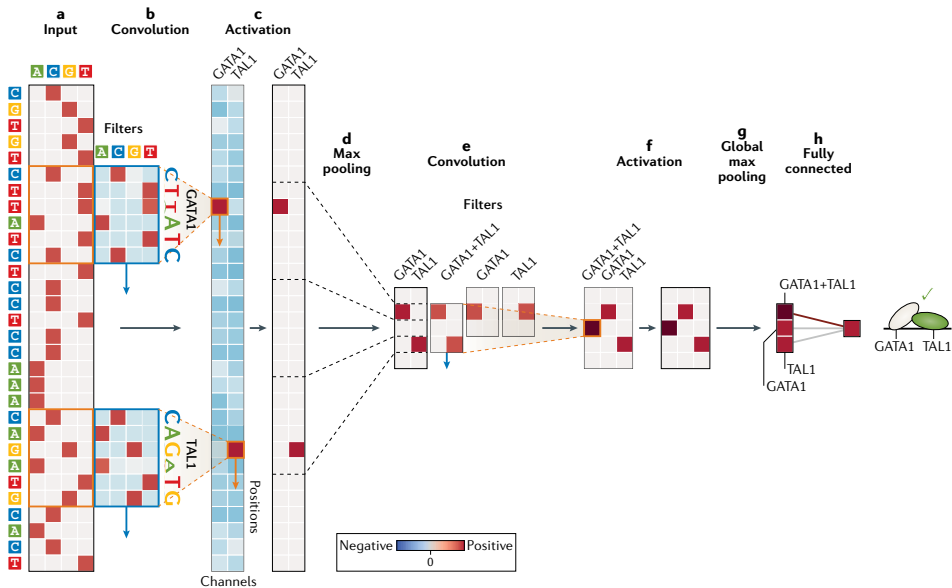
ResNet

Image generation by backprop to the input

Application in genomics (optional)

Summary

Predicting transcription factor binding sites (Eraslan et al., Nature Reviews Genetics 2019)



Summary

- Convolution operations capture translation invariant features by sliding a set of weight filters over the image
- Basic architecture involves alternating convolution and pooling layers followed by several dense layers for classification
- Understanding the schematics of ConvNets and neural network in general (detailed schema and compact schema)
- Batch normalization, data augmentation, inception, residual module with skip-connections help training deep ConvNets
- Pre-trained ConvNets can be used in generative model to generate new images

Objectives

Convolution

Padding

Stride

Dilated convolution

Multi-channels

Depthwise separable convolution

Pooling

Batch normalization

Basic ConvNet Architectures

LeNet

"We have to go deeper"

Data Augmentation

The ImageNet Dataset

AlexNet

GoogLeNet

ResNet

Image generation by backprop to the input

Application in genomics (optional)

Summary